

Why Economic Production in 'Open Source' is Free Software



For years, there has been rhetoric that 'free as in freedom' software misses the economic point and that it is only about the ideology. This prompted a community of individuals to coin a new term called open source that would focus on the economics minus the philosophy and politics of free software. This note is a study of the dialectics of free software, open source and proprietary software, aimed at understanding the underlying reasons for each approach, its dynamics and effects.

The GNU project was officially announced in September 1983. At the time the term 'free software' was coined, GNU had not yet evolved to a state in which it could take on the proprietary software world, and so the priority was to first promote the *idea* of software freedom. Over the subsequent decades, it became obvious to both academia and industry that community produced software or the bazaar model of software development generated more value than the proprietary model or the cathedral model. This was proved when Netscape announced that its source code would be released into the community, which proved to be economically viable, thus allowing the free software community to begin prioritising economic production. It was in the backdrop of this event that the Open Source

Initiative (OSI) was conceptualised and the definition for the term was coined by a community that wanted to keep the software production aspects of the free software philosophy, yet ensure the politics was out of sight.

At the outset, I would like to point out that the assumption that 'Free software is not about economics' is erroneous, and that economic relevance has always been a critical factor necessary for open source software as an idea, to stay relevant, survive and propagate.

Since the genesis of the GNU project in 1983 (which sprung up as a reaction to the proprietisation of software) free software has evolved in terms of development methodologies, community interactions and processes including industry-supported communities and business models. It was initially pointed out that free software would revolve around service-based models rather than software as a product, because software, being knowledge in a digital form, was a community produce and would have creators and users but not owners. However, it was not clear as to what the exact business models would be by which this economic production process would take place. The evolution of free software was hastened by the simple fact that several individuals and communities

have tirelessly produced a variety of software necessary for a solid foundation on which the movement is built. It was pointed out that GNU/Linux was developed by and for only geeks and hackers, and rightly so. And it was this strong point that ensured the software was secure and reliable, forcing industry majors to adopt it.

Currently, we are witnessing examples of corporations cannibalising the community and its produce. While open source did not directly contribute to this trend, the problem was aggravated by the fact that open source actively rejects the need to apply the philosophy and politics of free software – something that is needed to prevent the privatisation of community produce. An analogy to highlight the above point would be the 'mixed economy' model popular in India soon after its Independence, wherein the big business houses did not want to invest their substantial accumulated capital in setting up the infrastructural backbone needed for profitable industrialisation. They felt that they would not be able to compete in a free-market economy at that point and hence persuaded Jawaharlal Nehru to adopt the 'planned economy' approach, popularly referred to as the Bombay Plan. This plan involved the government initially investing public-community resources for a limited period to create the basic infrastructure for a stable economy and, subsequently, initiating the disinvestment of these public-community resources so that they could be cannibalised by the big corporations of both domestic and foreign origin.

Other notable examples in the software domain would be 'tivoisation' where free software is used in hardware but is locked in to prevent any kind of modification. 'Dual licensing' is the other method by which the mass of the software developed is obtained from community produce and specific features are built over this for enterprise customers using a non-free licence. These examples are representative.

The latest example is Microsoft's venture to open the code of .Net. The story that is not being told is that people